Programming Principles I

Week 3 · Class 5

Loops in C: while and for

Repeating work · while and for · ++, -- and +=

```
for (int i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
```

Quiz 1 is back

> **Three mistakes cost most of the marks in this room.**

None of them is hard. All three are **habits**.

Five minutes now, and they stop costing you marks.

Your mark is on the paper and in Blackboard. Read my comments at home, not now.

Mistake 1: %f always prints six decimals

```
double x = 6.25;
printf("%.2f\n", x);
printf("%f\n", x);
printf("%.0f\n", x);
```

Output

```
6.25
6.250000
6
```

%f — always **six**.

%.2f — you choose.

This one line cost marks on more papers than anything else in the quiz.

`%.0f` rounds. It does not cut.

x holds	%.2f	%f	%.0f
6.25	6.25	6.250000	6
2.75	2.75	2.750000	3

6.25 rounds **down** to 6. 2.75 rounds **up** to 3. It never just drops the digits.

Mistake 2: `scanf` needs `%lf` and `&`

What many of you wrote

```
double price;
scanf("%f", &price);

double kg;
scanf("%lf", kg);
```

What it has to be

```
double price;
scanf("%lf", &price);

double kg;
scanf("%lf", &kg);
```

For a `double`: `printf` takes `%f`, `scanf` takes `%lf`. And `scanf` always needs the `&`.

The compiler did tell you

```
double price;
scanf("%f", &price);

double kg;
scanf("%lf", kg);
```

gcc -Wall says

```
warning: format '%f' expects
argument of type 'float *',
but argument 2 has type 'double *'
```

It still builds.

It printed `Total: 0.00`.

Compile with `-Wall` and read what it says. This one was free to find.

Mistake 3: the small things that stop a program

1. Every statement ends with `;`
2. `main` ends with `return 0;` and a closing `}`
3. It is `#include <stdio.h>` — not `studio.h`, not `# insert`
4. C is case sensitive: `printf`, never `Printf`

> Each of these is half a point. The compiler finds every one of them for you.

All three come back today

> **Every loop you write today reads a number, counts, and prints a result.**

The same `scanf`. The same `%.2f`. The same `;`.

A quiz is small. Its job is to tell you **early** what to practise.

The practice test on Blackboard asks the same kind of question, as often as you like.

Today

1. Why loops exist
2. The four parts of a loop
3. The `while` loop
4. Tracing a loop with a pen
5. The loop that never stops
6. The `for` loop
7. `++` and `--`
8. `+=`, `-=`, `*=`, `/=`, `%=`

1 Why loops exist

From choosing to repeating

Last week your program learned to choose.

`if` this, `else` that. Then it ran to the end, once.

This week it learns to go **back** and do something **again**.

That is called a **loop**.

Print Hello five times

```c
#include <stdio.h>

int main(void) {
    printf("Hello\n");
    printf("Hello\n");
    printf("Hello\n");
    printf("Hello\n");
    printf("Hello\n");
    return 0;
}
```

Output

```
Hello
Hello
Hello
Hello
Hello
```

> Copy, paste, paste, paste, paste. It works.

A new request

QUESTION

Not 5 times.

1000 times.

What do you do now?

```
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
```

The same five Hellos, with a loop

```c
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 5; i++) {
        printf("Hello\n");
    }
    return 0;
}
```

Output

```
Hello
Hello
Hello
Hello
Hello
```

One `printf`.

The line above it says: do this 5 times.

From 5 to 1000

Copy and paste

```
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
printf("Hello\n");
```

Loop

```
for (int i = 0; i < 1000; i++) {
    printf("Hello\n");
}
```

Copy and paste: 995 more lines. Loop: change **one number**.

When you do not know the number

```c
#include <stdio.h>

int main(void) {
    int n;

    printf("How many times? ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Hello\n");
    }
    return 0;
}
```

Output

```
How many times? 3
Hello
Hello
Hello
```

The user decides how many.

You cannot copy and paste a number you do not know yet.

What loops are for

1. **Count:** 1, 2, 3, … 100
2. **Read many values:** 30 marks, one after another
3. **Search:** look at every item until you find it
4. **Ask again:** until the input is right

> Almost every real program has a loop in it.

2 The four parts of a loop

Every loop, in every language, has the same four parts

The four parts: start

Start	Check	Body	Update
Give a variable its first value. `int count = 1;`	A yes/no question, as in an `if`. `count <= 3`	The work you want repeated. `printf(...);`	Change the variable. `count++;`

> The start happens **once**.

The four parts: check

Start	Check	Body	Update
Give a variable its first value. `int count = 1;`	A yes/no question, as in an `if`. `count <= 3`	The work you want repeated. `printf(...);`	Change the variable. `count++;`

Yes: go on. No: the loop is over.

The four parts: body

Start	Check	Body	Update
Give a variable its first value.	A yes/no question, as in an `if`.	The work you want repeated.	Change the variable.
`int count = 1;`	`count <= 3`	`printf(...);`	`count++;`

This is why you wrote the loop.

The four parts: update

Start	Check	Body	Update
Give a variable its first value. `int count = 1;`	A yes/no question, as in an `if`. `count <= 3`	The work you want repeated. `printf(...);`	Change the variable. `count++;`

So that one day the answer to the check is **no**.

Start, check, body, update

Start	Check	Body	Update
Give a variable its first value. `int count = 1;`	A yes/no question, as in an `if`. `count <= 3`	The work you want repeated. `printf(...);`	Change the variable. `count++;`

> After the update: back to the **check**. Not back to the start.

One trip around is one round

> **Start → check → body → update → check → body → update → check …**

One trip around the circle is one **round**.

Books call it an **iteration**.

When a loop never stops

QUESTION

A loop never stops.

Which of the four parts is probably wrong?

3 The `while` loop

An `if` that keeps asking

`while` looks like `if`

if

```
if (condition) {
    // runs once
}
```

while

```
while (condition) {
    // runs again and again
}
```

> After an `if` block, C moves on. After a `while` block, C goes **back up** and asks again.

How a `while` loop runs

1. Check the condition.
2. If it is true, run the body.
3. Go back to the condition. Check again.
4. If it is false, leave the loop.

Type along: `count.c`

```c
#include <stdio.h>

int main(void) {
    int count = 1;

    while (count <= 3) {
        printf("%d\n", count);
        count++;
    }

    printf("Done\n");
    return 0;
}
```

The start

```c
#include <stdio.h>

int main(void) {
    int count = 1;

    while (count <= 3) {
        printf("%d\n", count);
        count++;
    }

    printf("Done\n");
    return 0;
}
```

One box, called `count`.

It holds 1.

This line runs **once**.

The check

```c
#include <stdio.h>

int main(void) {
    int count = 1;

    while (count <= 3) {
        printf("%d\n", count);
        count++;
    }

    printf("Done\n");
    return 0;
}
```

Is count at most 3?

Yes: run the body.

No: jump past the }.

The body

```c
#include <stdio.h>

int main(void) {
    int count = 1;

    while (count <= 3) {
        printf("%d\n", count);
        count++;
    }

    printf("Done\n");
    return 0;
}
```

The lines inside { }.

They run once in every round.

The last line of this body is also the update.

The update

```c
#include <stdio.h>

int main(void) {
    int count = 1;

    while (count <= 3) {
        printf("%d\n", count);
        count++;
    }

    printf("Done\n");
    return 0;
}
```

`count++` means: add 1 to `count`.

Short for `count = count + 1`.

Without it the loop never stops.

The closing brace

```c
#include <stdio.h>

int main(void) {
    int count = 1;

    while (count <= 3) {
        printf("%d\n", count);
        count++;
    }

    printf("Done\n");
    return 0;
}
```

At the } C goes **back up** to the check.

Not to `int count = 1;`.

After the loop

```c
#include <stdio.h>

int main(void) {
    int count = 1;

    while (count <= 3) {
        printf("%d\n", count);
        count++;
    }

    printf("Done\n");
    return 0;
}
```

Output

```
1
2
3
Done
```

Done is outside the loop.

It is printed once.

Compile it and run it

```
gcc -Wall count.c -o count
./count
```

Output

```
1
2
3
Done
```

> In Windows PowerShell the second line is `.\count`

Walk through it: check 1

```
int count = 1;

while (count <= 3) {
    printf("%d\n", count);
    count++;
}

printf("Done\n");
```

count	count <= 3	prints	then count
1	true	1	2

count is 1. Is 1 at most 3? Yes. Print 1. count becomes 2.

Walk through it: check 2

```
int count = 1;

while (count <= 3) {
    printf("%d\n", count);
    count++;
}

printf("Done\n");
```

count	count <= 3	prints	then count
1	true	1	2
2	**true**	**2**	**3**

Back up. `count` is 2. Yes. Print 2. `count` becomes 3.

Walk through it: check 3

```
int count = 1;

while (count <= 3) {
    printf("%d\n", count);
    count++;
}

printf("Done\n");
```

count	count <= 3	prints	then count
1	true	1	2
2	true	2	3
3	**true**	**3**	**4**

Back up. `count` is 3. Yes. Print 3. `count` becomes 4.

Walk through it: check 4

```
int count = 1;

while (count <= 3) {
    printf("%d\n", count);
    count++;
}

printf("Done\n");
```

count	count <= 3	prints	then count
1	true	1	2
2	true	2	3
3	true	3	4
4	**false**		

Back up. `count` is 4. Is 4 at most 3? **No.** Leave the loop. Done.

Change one thing

PREDICT

`count` starts at **10**.

What is printed?

```
int count = 10;

while (count <= 3) {
    printf("%d\n", count);
    count++;
}

printf("Done\n");
```

Change one thing: the answer

PREDICT

`count` starts at **10**.

What is printed?

```
int count = 10;

while (count <= 3) {
    printf("%d\n", count);
    count++;
}

printf("Done\n");
```

ANSWER

Only Done. Is 10 at most 3? No, at the very first check.

Zero rounds is normal

A `while` loop checks before every round.

Also before the **first** one.

If the check is false at the start, the body runs **zero times**.

That is not an error. The program just goes on.

One more: counting down

```c
#include <stdio.h>

int main(void) {
    int n = 5;

    while (n > 0) {
        printf("%d\n", n);
        n--;
    }

    printf("Liftoff!\n");
    return 0;
}
```

Output

```
5
4
3
2
1
Liftoff!
```

`n--` takes 1 away from n.

4

Tracing a loop with a pen

Do not guess the output. Track the values.

Why trace?

> **On a quiz you have no compiler. You have a pen.**

Draw a table.

One row per round.

Write down the variable, the check, and what is printed.

Trace it on paper

YOUR TURN

What does it print?

What is `x` **after** the loop?

2 minutes. Pen and paper.

```
int x = 1;

while (x <= 4) {
    printf("%d ", x);
    x = x + 1;
}
```

The trace table: row 1

```
int x = 1;

while (x <= 4) {
    printf("%d ", x);
    x = x + 1;
}
```

round	x	x <= 4	prints	then x
1	1	true	1	2

Round 1: x is 1. 1 <= 4 is true. Print 1. x becomes 2.

The trace table: row 2

```
int x = 1;

while (x <= 4) {
    printf("%d ", x);
    x = x + 1;
}
```

round	x	x <= 4	prints	then x
1	1	true	1	2
2	**2**	**true**	**2**	**3**

Round 2: x is 2. True. Print 2. x becomes 3.

The trace table: row 3

```
int x = 1;

while (x <= 4) {
    printf("%d ", x);
    x = x + 1;
}
```

round	x	x <= 4	prints	then x
1	1	true	1	2
2	2	true	2	3
3	**3**	**true**	**3**	**4**

Round 3: x is 3. True. Print 3. x becomes 4.

The trace table: row 4

```
int x = 1;

while (x <= 4) {
    printf("%d ", x);
    x = x + 1;
}
```

round	x	x <= 4	prints	then x
1	1	true	1	2
2	2	true	2	3
3	3	true	3	4
4	**4**	**true**	**4**	**5**

Round 4: x is 4. 4 <= 4 is true, because of the =. Print 4. x becomes 5.

The trace table: row 5

```
int x = 1;

while (x <= 4) {
    printf("%d ", x);
    x = x + 1;
}
```

round	x	x <= 4	prints	then x
1	1	true	1	2
2	2	true	2	3
3	3	true	3	4
4	4	true	4	5
	5	**false**		

x is 5. 5 <= 4 is **false**. The loop ends. Nothing is printed.

What the table tells you

Output: `1 2 3 4` After the loop: `x` is 5

The body ran **4** times.

The check ran **5** times: four times true, one time false.

The check always runs one more time than the body.

A favourite quiz question

What is x after the loop?

It is **not** the last number printed.

It is the **first number that failed the check**.

x had to become 5, or x <= 4 would still be true.

5

The loop that never stops

You will write one by accident. Everyone does.

The update is missing

```
int count = 1;

while (count <= 3) {
    printf("%d\n", count);
    // count never changes
}

printf("Done\n");
```

count is 1. For ever.

1 <= 3 is true. For ever.

It prints 1, 1, 1, 1, ...

The compiler says nothing

> **`gcc -Wall` prints no warning for this program.**

A loop that never ends is perfectly spelled C.

The compiler checks your **spelling**, not your **thinking**.

How to stop a running program: Ctrl + C

1. Click once inside the terminal.
2. Hold the **Ctrl** key.
3. Press **C**. Once.

Windows, Mac, Linux: the same. On a Mac it is **Control**, not Command.

Your turn: break it, stop it, fix it

1. Open `count.c`. Put `//` in front of `count++;`
2. Compile with `-Wall`. Any warning?
3. Run it. A wall of 1s.
4. Stop it with **Ctrl + C**.
5. Take the `//` away. Compile. Run.

Never stops / stops

Never stops

```
while (count <= 3) {
    printf("%d\n", count);
    // count never changes
}
```

Fixed

```
while (count <= 3) {
    printf("%d\n", count);
    count++;
}
```

The habit: **before** you run a loop, put your finger on the update.

A silent cousin: the semicolon

```
int count = 1;

while (count <= 3);
{
    printf("%d\n", count);
    count++;
}
```

The `;` is an **empty body**.

The loop repeats nothing, for ever. The screen stays empty.

Program frozen? Ctrl + C, then look for this `;`

6 The for loop

Start, check and update on one line

Three parts in three places

```
int count = 1;

while (count <= 3) {
    printf("%d\n", count);
    count++;
}
```

The start is above the loop.

The check is in the brackets.

The update is at the bottom. Easy to forget.

The shape of a `for` loop

```
for (start; check; update) {
    body
}
```

Three parts, separated by **semicolons**, not commas. No semicolon after the).

When each part runs

`int i = 1`	`i <= 5`	`i++`
Start runs **once**, at the beginning	**Check** tested **before** each round	**Update** runs **after** each round

Read it aloud: start `i` at 1; go on while `i` is at most 5; after each round, add 1 to `i`.

Type along: `for.c`

```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        printf("%d ", i);
    }

    printf("\n");
    return 0;
}
```

Output

```
1 2 3 4 5
```

The order inside a `for`: round 1

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

i	i <= 5	prints	then i
1	true	1	2

Start: `i` is 1. Check: true. Body: print 1. Update: `i` becomes 2.

The order inside a `for`: round 2

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

i	i <= 5	prints	then i
1	true	1	2
2	**true**	**2**	**3**

Check again: `i` is 2. True. Print 2. `i` becomes 3.

The order inside a `for`: the whole table

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

i	i <= 5	prints	then i
1	true	1	2
2	true	2	3
3	true	3	4
4	true	4	5
5	true	5	6
6	**false**		

... until `i` is 6. `6 <= 5` is false. The loop ends.

The title slide, answered

It prints 1, 2, 3, 4, 5.

Which part changes `i` after each round?

The third one: `i++`.

`i` lives only inside the loop

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}

printf("%d\n", i);
```

The compiler says

```
error: 'i' undeclared
```

`i` was made on the `for` line.

When the loop ends, the box is gone.

Make it count to 10

YOUR TURN

Change `for.c` so that it prints 1 to 10.

How many characters did you change?

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

The body can use the counter

```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        printf("%d squared is %d\n", i, i * i);
    }
    return 0;
}
```

Output

```
1 squared is 1
2 squared is 4
3 squared is 9
4 squared is 16
5 squared is 25
```

The same loop, two ways

while

```
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}
```

for

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

Both print 1 2 3 4 5. The same four parts, in different places.

Which one do I use?

for	while
You **know** how many rounds. 5 times. n times. From 1 to 100.	You **do not know** how many. Keep going until something happens.

`for` or `while`?

QUESTION

Print the names of the 12 months.

Keep asking for the password until it is right.

7

++ and - -

Add one. Take one away.

Short forms for "change by one"

You write	It means	Say it as
`i++;`	`i = i + 1;`	add 1 to i
`++i;`	`i = i + 1;`	add 1 to i
`j--;`	`j = j - 1;`	take 1 from j
`--j;`	`j = j - 1;`	take 1 from j

On a line of its own, `i++` and `++i` do exactly the same thing.

Counting down with for

```c
#include <stdio.h>

int main(void) {
    for (int i = 5; i >= 1; i--) {
        printf("%d ", i);
    }

    printf("\n");
    return 0;
}
```

Output

```
5 4 3 2 1
```

Up and down: all three parts change

Count up

```
for (int i = 1; i <= 5; i++)
```

Count down

```
for (int i = 5; i >= 1; i--)
```

Start at the big number. Check with `>=`. Update with `--`. Change only one or two, and you have a bug.

A habit gone wrong

PREDICT

Start at 5.

Check `i >= 1`.

By habit: `i++`.

What happens?

```
for (int i = 5; i >= 1; i++) {
    printf("%d ", i);
}
```

A habit gone wrong: the answer

PREDICT

Start at 5.

Check `i >= 1`.

By habit: `i++`.

What happens?

```
for (int i = 5; i >= 1; i++) {
    printf("%d ", i);
}
```

ANSWER

5 6 7 8 9 … `i >= 1` stays true.
It never stops. Ctrl + C.

a++ and ++a

On a line of their own: the same.

The difference shows only when you **use the value in the same line**.

a++: use, then add

```
int a = 5;
int b = a++;
```

The ++ is **after** the a.

First use a: b gets **5**.

Then add: a becomes **6**.

++a: add, then use

```
int a = 5;
int b = ++a;
```

The ++ is **before** the a.

First add: a becomes **6**.

Then use a: b gets **6**.

Hands up

PREDICT

What is printed?

`5 6` or `6 6` or `5 5`

```
int x = 5;

printf("%d ", x++);
printf("%d\n", x);
```

Hands up: the answer

PREDICT

What is printed?

`5 6` or `6 6` or `5 5`

```
int x = 5;

printf("%d ", x++);
printf("%d\n", x);
```

ANSWER

`5 6`. The first `printf` gets the old value, 5. Then `x` becomes 6.

The rule for this course

> **Write `i++` on a line of its own, or as the third part of a `for`.**

You must be able to **read** `b = a++`. It can be on a quiz.

You never need to **write** it.

Never write `i++ + ++i`. Even the compiler objects: `operation on 'i' may be undefined`.

8 += and its family

A shorter way to write `x = x + 3`

Take, add, put back

Long form

```
x = x + 3;
```

Short form

```
x += 3;
```

> Take what is in the box, add 3, put it back. Say it as: **add 3 to** x.

All five

Short form	Long form	Say it as
`x += 3;`	`x = x + 3;`	add 3 to x
`x -= 2;`	`x = x - 2;`	take 2 from x
`x *= 4;`	`x = x * 4;`	multiply x by 4
`x /= 5;`	`x = x / 5;`	divide x by 5
`x %= 2;`	`x = x % 2;`	keep the remainder

And now you see what x++ is: `x += 1`, which is `x = x + 1`.

Row by row

PREDICT

x starts at 10.

What is x after each line?

```
int x = 10;

x += 3;
x -= 2;
x *= 4;
x /= 5;
x %= 2;
```

Row by row: the answer

```
int x = 10;

x += 3;
x -= 2;
x *= 4;
x /= 5;
x %= 2;
```

line	x is now
`x += 3`	13
`x -= 2`	11
`x *= 4`	44
`x /= 5`	8
`x %= 2`	0

`44 / 5` is 8, not 8.8: `int` divided by `int` throws the decimals away. `8 % 2` is 0.

The update does not have to be ++

```c
#include <stdio.h>

int main(void) {
    for (int i = 0; i <= 10; i += 2) {
        printf("%d ", i);
    }

    printf("\n");
    return 0;
}
```

Output

```
0 2 4 6 8 10
```

A box that collects

```c
#include <stdio.h>

int main(void) {
    int sum = 0;

    for (int i = 1; i <= 4; i++) {
        sum += i;
    }

    printf("%d\n", sum);
    return 0;
}
```

Output

```
10
```

`sum` starts at 0.

Each round: add `i` to `sum`.

Trace the sum: round 1

```
int sum = 0;

for (int i = 1; i <= 4; i++) {
    sum += i;
}

printf("%d\n", sum);
```

i	sum += i	sum
1	0 + 1	1

Trace the sum: round 2

```
int sum = 0;

for (int i = 1; i <= 4; i++) {
    sum += i;
}

printf("%d\n", sum);
```

i	sum += i	sum
1	0 + 1	1
2	**1 + 2**	**3**

Trace the sum: round 3

```
int sum = 0;

for (int i = 1; i <= 4; i++) {
    sum += i;
}

printf("%d\n", sum);
```

i	sum += i	sum
1	0 + 1	1
2	1 + 2	3
3	**3 + 3**	**6**

Trace the sum: round 4

```
int sum = 0;

for (int i = 1; i <= 4; i++) {
    sum += i;
}

printf("%d\n", sum);
```

i	sum += i	sum
1	0 + 1	1
2	1 + 2	3
3	3 + 3	6
4	**6 + 4**	**10**

Prints `10`.

The accumulator

> **A box that collects a little more in every round.**

It is the most useful idea in all of loops.

It is where **Friday** starts.

Today, in six sentences

1. A loop has four parts: **start, check, body, update**.
2. `while` puts them in three places.
3. `for` puts three of them on one line.
4. No update: the loop never stops. **Ctrl + C**.
5. `i++` adds one. `sum += i` adds `i`.
6. Do not guess the output. **Trace it.**

Before Friday

1. **Knowledge check 5** opens tonight on Blackboard.
2. **Vote** on the number of quizzes. It closes Thursday 23:59.
3. Type `count.c` and `for.c` again, without looking.
4. Make each of them count from 10 down to 1.
5. Download **Class 5 code** from Blackboard and do the "Try" lists.

Exit ticket

QUESTION

Name the four parts of a loop.